ELARA: Energy-Efficient and Latency-Aware Microservice Orchestration in Space Computing Power Networks

Anonymous Authors

***Abstract*—The rapid advancement of the Space Computing Power Network (SCPN), comprising numerous low Earth orbit (LEO) satellites, has enabled the execution of complex in-orbit observation and computation tasks. To facilitate this, microservice architecture decomposes large, monolithic remote sensing and image processing applications into independent, schedulable, and reusable services deployed in a distributed manner across LEO satellites. However, integrating these capabilities presents significant challenges arising from the limited computing resources, constrained power provisions of individual satellites, and the dynamic nature of space network topology and channel conditions. This paper addresses these challenges by proposing an energy-aware service routing algorithm for microservices within the SCPN. The algorithm aims to minimize the energy consumed by the procedures of processing and transmitting service data in both computation and communication. By modeling the service routing problem as a Directed Steiner Tree (DST) and implementing heuristic-based solutions, our algorithm effectively responds to the diverse service patterns requested by terrestrial clients. Comprehensive experimental results demonstrate that our approach significantly outperforms existing methods in terms of energy efficiency, robustness, stability, and adaptability.**

***Index Terms*—Energy, Microservice, Routing, Space Network**

I. Introduction

Recent advances in low Earth orbit (LEO) mega-constellations are transforming satellites from bent-pipe communication relays into distributed computing nodes in space. These large-scale constellations, comprising hundreds to thousands of satellites connected by inter-satellite links (ISLs) and equipped with onboard processors and storage resources, enable global connectivity and near-data processing for time-sensitive applications [1]–[3]. This evolution is particularly critical for Earth observation, waether forecasting, environmental monitoring, and remote sensing analytics, where massive volumes of images and multi-modal data are continuously generated in orbit [4]–[6]. Directly downloading such raw data to terrestrial data centers for analysis will consume scarce satellite-to-ground bandwidth and exacerbate link congestion, especially under limited contact windows and unfavorable channel conditions. [7], [8]. As a result, space computing power networks (SCPNs), which pool sensing, communication, and computing resources across LEO satellites, have emerged as a promising infrastructure for onboard intelligent services [9], [10].

Promising but challenging, running these complex monolithic applications on a single LEO satellite remains infeasible due to stringent resource constraints, e.g., limited computing, storage, energy provisions, and thermal dissipation [11], [12]. Microservice architecture provides an effective solution by decomposing these remote sensing and AI applications into fine-grained, loosely coupled, and reusable microservices, each encapsulating a distinct functional module [13]–[16]. By packaging them into lightweight containers and deploying them across interconnected satellites, this architecture alleviates individual satellite resource limitations and enables onboard cooperative execution of complex applications. Furthermore, upon detecting hotspot microservices, the system can dynamically scale their replicas and adjust deployment strategies to ensure robust and efficient service provisioning.

Typically, a single request initiated by a terrestrial user involves a subset of microservices. The dynamic and time-varying constellation topology can affect or even interrupt the data transmission process because of unstable inter-plane ISLs. Moreover, the harware heterogeneity and limited onboard power supply further complicate the scheduling decisions at each microservice execution stage.

To enable energy-efficient and low-latency service provision in SCPNs, there are several major challenges must be addressed. First, the satellite network topology is predictable but highly time-varying. Intra-plane ISLs are relatively stable, whereas inter-plane ISLs are sensitive to satellite motion and change frequently across slots. Consequently, routing intermediate data along a single static path often suffers from link disruptions, congestion, or degraded transmission rates. Second, serving satellite selection and ISL routing are tightly coupled. Selecting a nearby replica may reduce transmission delay but lead to long computation queuing time on overloaded satellites, while choosing an idle satellite can incur high multi-hop communication costs. This coupling is further complicated by heterogeneous computing capacities, and energy constraints [17]–[19]. Third, microservice replica deployment cannot remain static. As request distributions, satellite workloads, and network topology evolve, previously optimal placements quickly become inefficient. However, frequent redeployment is costly due to expensive service image transfers over ISLs and the enormous decision space [20].

Existing studies have extensively investigated satellite edge computing, in-orbit computation, service placement, and NFV-enabled service chaining in the context of SCPN [7], [8], [20]–[24]. Nevertheless, most of them fail to fully capture the continuous execution of microservice requests and ignore the joint optimization of service routing and replica deployment

under time-varying communication conditions and heterogeneous onboard hardware environments. In particular, many satellite-oriented service and function execution models rely on unchanged constellation snapshots, implicitly assuming that each task can be completed within a single period during which link states remain unchanged. While this abstraction simplifies modeling and algorithm design, it deviates from practical in-orbit service execution, where data routing, computation, and scheduling could frequently span multiple time slots [25]. Although terrestrial microservice routing studies have explored the joint optimization of service deployment and request routing [26], [27], their assumptions of stable networks, abundant power supply, and fixed edge infrastructure are not directly applicable to the SCPNs.

A. Contributions

To address the aforementioned challenges, we propose ELARA, an energy-efficient and latency-aware microservice orchestration framework for SCPNs. ELARA jointly optimizes routing and deployment decisions for distributed microservice-based applications in spatio-temporally dynamic SCPN environments. Specifically, upon a request arrives at an arbitrary continuous time, ELARA partitions the entire execution process into multiple sequential stages. For each stage, after completing the current service, ELARA first selects the next computing node from satellites hosting replicas of the subsequent microservice, while considering the volume of intermediate data, real-time onboard computing capacities, and current ISL states. ELARA then runs a service routing algorithm based on the source and destination satellites, available bandwidths, and ISL congestion levels, generating a set of parallel paths to accelerate data transmission and improve robustness against link disruptions. Finally, ELARA periodically adjusts microservice deployments according to historical service invocation patterns and real-time computing and communication conditions of the SCPN. Built upon this design, ELARA introduces a continuous-time cross-slot service provisioning mechanism along with an adaptive routing-and-redeployment strategy, enabling joint optimization of end-to-end latency and energy consumption under dynamic constellation topologies and stringent onboard resource constraints.

In summary, the main contributions are as follows:

1) We identify and formulate the microservice-based in-orbit service provisioning problem in SCPNs from an application-layer perspective. In particular, we characterize the intertwined challenges of service routing and replica deployment under time-varying constellation topology, heterogeneous onboard computing capacities, and stringent resource constraints. To address these challenges, we propose ELARA, an energy-efficient and latency-aware orchestration framework for microservice execution and deployment across multiple satellites.
2) We design a learning-based service orchestration algorithm that integrates GNN-enhanced Proximal Policy Optimization (PPO) with time-slot-aware adaptive routing. ELARA employs a graph neural network (GNN) to jointly encode satellite computing states, replica deployment, ISL capacity, and service calling information, enabling the PPO agent to learn effective next-hop serving satellite selection policies that adapt to computation heterogeneity and topology dynamics. For data transfer, it further adopts a time-slot-aware min-cost max-flow algorithm to generate parallel and robust routing paths across dynamic LEO snapshots.
3) We propose a multi-armed bandit-based algorithm for adaptive replica placement. By collecting request patterns over a few past time slots along with current onboard computing and ISL conditions, ELARA dynamically adjusts the number of replicas and deployments of hotspot microservices, whose decisions are made at the orbital level rather than by each individual satellite for reducing learning overhead and decision complexity.
4) We conduct extensive simulations under dynamic LEO constellations and diverse microservice workloads to evaluate ELARA's performance. The results demonstrate that ELARA can effectively reduces end-to-end latency, energy consumption, and deadline violations compared with representative baselines, while validating the benefits of joint service orchestration, adaptive routing, and replica deployment.

The remainder of this paper is organized as follows. Sec. II reviews the related works. Sec. III presents the system models. The problem formulation is introduced in Sec. IV. Sec. V details the proposed ELARA framework. Sec. VI shows and analyses the Simulation results. Sec. VII draws the conclusion.

II. Related Work

Recent advancements in satellite edge computing and SCPNs have supported near-data processing over LEO constellations with onboard computation and ISL connectivity [9]–[11], [28]. Zhang et al. [29] investigated energy-efficient peer offloading among satellites, where computation tasks are cooperatively migrated to reduce energy consumption. Leyva-Mayorga et al. [7] studied satellite edge computing for real-time very-high-resolution Earth observation, and Zhai et al. [6] proposed SECO to jointly optimize multi-satellite observation scheduling, routing, and computation node selection. These efforts mainly focus on task-level offloading, mission-specific processing, or computation-communication resource allocation. Microservice-oriented service provision, however, introduces finer-grained execution dependencies across distributed replicas, where intermediate data transmission and onboard computation are tightly coupled under dynamic SCPNs.

Service routing and task scheduling in satellite networks have also been studied from various of perspectives. In [19], the author proposed a computing-aware routing framework that integrates transmission and computation decisions for remote sensing tasks in LEO satellite networks. Xia et al. [22] studied service chaining in NFV-enabled satellite edge networks, while Abreha et al. [23] considered fairness-aware VNF mapping and scheduling for mission-critical services. For improve routing effiency and avoiding link congestion, He et

al. [24] investigated throughput-oriented routing for microservice flows in LEO satellite networks. Moreover, some works further model dynamic networks with time-expanded graphs or optimize routing over time-varying LEO topologies [25], [30], [31]. However, most of them either optimize communication paths alone or consider computation-node selection within a given stable topology snapshot or a pre-defined topology sequence. They fails to capture the joint impact of broken inter-plane ISLs, heterogeneous serving node computing capacity, and time-varying SCPNs for microservice requests.

Microservice deployment combined with request execution have been extensively studied in terrestrial edge systems. Yu et al. [26] formulated the joint optimization of request routing and instance placement in microservice systems, and Peng et al. [27] designed algorithms for joint service deployment and request routing in mobile edge computing. For satellite-oriented environments, Gao et al. [21] studied joint server and service selection in satellite-terrestrial integrated edge computing networks, and Yu et al. [20] proposed a robust reinforcement learning approach for microservice deployment in SCPNs. Nevertheless, existing deployment methods either are optimized for fixed terrestrial scenarios and not suit for time-varying SCPNs or lead to large satellite-service co-decision spaces. As a result, how to coordinate microservice deployment with request-time orchestration under time-varying satellite computing conditions and ISL states remains insufficiently explored and addressed.

III. System Model

We consider an SCPN built upon a Walker-Delta LEO constellation, where satellites are connected through time-varying laser ISLs and equipped with constrained onboard computing and storage resources. Microservice-based applications are deployed as stateless replicas across satellites, allowing each request to be collaboratively executed through successive communication and computation stages. This section models the constellation topology, ISL communication, onboard computation, microservice requests, cross-slot execution cost, and replica migration process.

A. LEO Constellation and Time-slotted Topology

A typical Walker-Delta constellation is parameterized by $\mathcal{W} = (A/P/G, H, I)$, where A is the total number of satellites, P is the number of orbital planes, G is the inter-plane phasing factor, H is the orbital altitude, and I is the inclination. Consequently, each orbital plane accommodates $N = A/P$ satellites. The set of satellite nodes in the constellation is formally defined as $\mathcal{V} = \{v_{p,n} \mid p = 1, \ldots, P,\ n = 1, \ldots, N\}$, where p denotes the orbital plane index, n denotes the intra-plane position index, To capture the high dynamism of the satellite network, we discretize the constellation's orbital period into a set of time slots, as $\mathcal{T} = \{0, 1, \ldots, T-1\}$, with a uniform slot duration Δ [32]. For any continuous-time instant τ, the corresponding time slot index is mapped as, $s(\tau) = \lfloor \tau/\Delta \rfloor \bmod T$. Adopting a snapshot-based modeling approach, we assume that the network state, including satellite positions, ISL availability, and capacity, remains unchanged within a single time slot and only evolves while the slot is switching. Thus, the time-varying network topology during slot $t \in \mathcal{T}$ is formally defined as a dynamic and time-dependent graph $\mathcal{G}(t) = \big(\mathcal{V}, \mathcal{E}(t)\big)$, where $\mathcal{E}(t)$ represents the set of feasible ISLs at slot t.

B. ISL Communication Model

We assume that each satellite is equipped with four laser optical terminals: two positioned along the roll axis for intra-plane ISLs, and two along the pitch axis for inter-plane ISLs.

a) ISL Connectivity Constraints: Each satellite $v_{p,n}$ maintains two intra-plane ISLs with its adjacent neighbors v_{p,n^-} and v_{p,n^+}, where $n^{\pm} = ((n \pm 1) \bmod N) + 1$. Because satellites in the same plane share identical angular velocities, their relative distances remain time-invariant, yielding highly stable communication links.

Conversely, inter-plane ISLs are dynamic. Constrained by the physical limits of onboard laser communication terminals (LCTs), the existence of a inter-plane link $e = (u, v)$ at slot t is jointly affected by line-of-sight (LoS) distance, mechanical tracking thresholds, and polar exclusions, formulated as:

$$e \in \mathcal{E}(t) \iff \begin{cases} d_{u,v}(t) \le d_{\max}, \\ \theta_{u,v}(t) \le \theta_{\max}, \\ \max\big(|\varphi_u(t)|, |\varphi_v(t)|\big) \le \varphi_{\max}. \end{cases} \tag{1}$$

where the LoS distance $d_{u,v}(t)$ and relative tracking angle $\theta_{u,v}(t)$ are bounded by the terrestrial occlusion limit $d_{\max}$ and the hardware threshold $\theta_{\max}$, respectively. Furthermore, the third constraint enforces a polar exclusion zone at latitude boundary $\varphi_{\max}$ (e.g., 70°) through proactively tearing down the inter-plane links, which effectively circumvents the severe pointing dynamics and extreme Doppler shifts inherent to polar regions.

b) ISL Capacity Model: Unlike terrestrial networks with stable capacities, the transmission rate of laser ISLs is distance-dependent due to geometric propagation loss. Based on the Free-Space Optical (FSO) communication model, the received power $P_e^{\text{rx}}(t)$ at satellite v transmitted from satellite u is formalized as:

$$P_e^{\text{rx}}(t) = P^{\text{tx}} G^{\text{tx}} G^{\text{rx}} \eta_{\text{tx}} \eta_{\text{rx}} \left(\frac{\lambda}{4\pi d_{u,v}(t)} \right)^2, \tag{2}$$

where P^{tx} is the laser transmit power, G^{tx} and G^{rx} represent the optical gains of the transmitter and receiver apertures respectively, λ is the laser wavelength, and $\eta_{\text{tx}}, \eta_{\text{rx}}$ denote the optical efficiency factors. The real-time Signal-to-Noise Ratio (SNR) of the link e at time slot t is given as:

$$\gamma_e(t) = \frac{\left(R_{\text{pd}} \cdot P_e^{\text{rx}}(t)\right)^2}{\sigma_{\text{sh}}^2 + \sigma_{\text{th}}^2}, \tag{3}$$

where R_{pd} is the photodiode responsivity, while σ_{sh}^2 and σ_{th}^2 denote the shot and thermal noise variances, respectively. According to the Shannon-Hartley theorem, the achievable transmission capacity $R_e^0(t)$ is constrained by:

$$R_e^0(t) = \mathrm{B}_e \log_2\left(1 + \gamma_e(t)\right), \tag{4}$$

where B_e is the modulation bandwidth.

c) ISL Delay Model: The ideal rate $R_e^0(t)$ in (4) could be degraded in practice by transient channel fading and traffic contention. To capture these factors, we introduce a scaling factor $\eta_e(t) \in (0,1]$ to formulate the effective data rate as $R_e(t) = \eta_e(t) R_e^0(t)$. When transmitting a data block of size B over link $e = (u, v)$ during slot t, the aggregate link-level communication delay comprises queuing, transmission, and propagation delays, formulated as:

$$T_e^{\mathrm{cm}}(B,t) = \nu_e^{\mathrm{qe}}(t) + \nu_e^{\mathrm{tr}}(t) + \nu_e^{\mathrm{pr}}(t). \tag{5}$$

Here, $\nu_e^{\mathrm{qe}}(t)$ denotes the queuing delay, which can be dynamically estimated via lightweight queue-length probing. The transmission delay is given by $\nu_e^{\mathrm{tr}}(t) = B/R_e(t)$, and the propagation delay is $\nu_e^{\mathrm{pr}}(t) = d_e(t)/c$, with c being the speed of light in vacuum.

C. Onboard Computing Model

Let F_v^0 denote the ideal computing capacity of satellite v. In practice, the available computational resources are often constrained by background payload tasks, operating system overhead, and thermal throttling. To capture these hardware and software limitations, we introduce a computational efficiency factor $\xi_v(t) \in (0,1]$ and define the effective dynamic computing capacity as $F_v(t) = \xi_v(t) F_v^0$.

Let $\nu_v^{\mathrm{qe}}(t)$ denote the queuing delay before scheduling microservice m is executed on satellite v, which increases with the pending task queue length at slot t. The total computation delay is modeled as:

$$T_m^{\mathrm{cp}}(v,t) = \nu_v^{\mathrm{qe}}(t) + \nu_v^{\mathrm{cp}}(t), \tag{6}$$

where $\nu_v^{\mathrm{cp}}(t) = C_m/F_v(t)$ represents the actual execution delay, and C_m is the number of CPU cycles required by microservice m. Correspondingly, the computational energy consumption is defined as:

$$E_m^{\mathrm{cp}}(v,t) = P_v^{\mathrm{cp}} \frac{C_m}{F_v(t)}, \tag{7}$$

where P_v^{cp} is the operational computing power of the satellite.

D. LEO Microservice and Service Request Models

To facilitate flexible application management within the SCPN, complex applications are decoupled into a set of fine-grained, loosely coupled microservices, denoted by $\mathcal{M} = \{1, 2, \ldots, M\}$. Each microservice $m \in \mathcal{M}$ is characterized by its computational demand C_m (in CPU cycles), container image size I_m, and storage requirement S_m.

We define a binary indication variable $x_{m,v}(t) \in \{0,1\}$, which equals to 1 if m is deployed on v during slot t. Accordingly, the subset of satellite nodes hosting a valid replica of microservice m at slot t is denoted as $\mathcal{R}_m(t) = \{v \in \mathcal{V} \mid x_{m,v}(t) = 1\}$. To ensure service availability while preventing excessive resource contention, the total number of deployed replicas for any microservice should be bounded within predefined thresholds, $r_m^{\min}$ and $r_m^{\max}$:

$$r_m^{\min} \le \sum_{v \in \mathcal{V}} x_{m,v}(t) \le r_m^{\max}, \quad \forall m \in \mathcal{M}. \tag{8}$$

Furthermore, the aggregate storage footprint of all replicas hosted on a single satellite must not exceed its onboard storage capacity $S_v^{\max}$:

$$\sum_{m \in \mathcal{M}} x_{m,v}(t) S_m \le S_v^{\max}, \quad \forall v \in \mathcal{V}. \tag{9}$$

Let $r = \langle v_s, m_1, m_2, \ldots, m_L, v_d, \tau_{\mathrm{in}} \rangle$ denote an incoming request with a deadline Z_r. Here, v_s and v_d are the source and destination satellites, m_i is the i-th microservice in the sequential execution chain, and τ_{in} is the continuous arrival time. Let $B_{r,i}$ be the data volume that should be transmitted before invoking m_i, while $B_{r,L+1}$ denotes the final output routed to v_d.

a) Cross-slot Service Routing Latency and Energy Model: Since requests may arrive at arbitrary instants while ISL topology and rates change at slot boundaries, ELARA tracks each service chain in continuous time. Before executing m_i, the input data $B_{r,i}$ stored at satellite $u_{r,i}$ should be routed to a satellite hosting a valid replica of m_i. We introduce a binary decision variable $y_{r,i,v} \in \{0,1\}$ for node selection, which equals 1 if m_i is scheduled to satellite v. Let $v_{r,i}$ be the selected satellite with $y_{r,i,v_{r,i}} = 1$. For the i-th service routing operation, the source and destination satellites are denoted by $u_{r,i}$ and $v_{r,i}$, respectively.

Since the endpoints may be non-adjacent and transmission may start near a slot boundary, one routing operation can span $H \ge 1$ phases over time-varying multi-hop routes. For phase h, let $\tau_{r,i}^h$ be its start time, $t_h = s(\tau_{r,i}^h)$ the active topology slot, $\bar{\Delta}_h$ the remaining slot time, and $B_{r,i}^h$ the residual data, with $B_{r,i}^1 = B_{r,i}$. Let $\mathcal{P}_{r,i}^h$ be the feasible and active end-to-end path set from $u_{r,i}$ to $v_{r,i}$ in slot t_h, where each path $p \in \mathcal{P}_{r,i}^h$ is an ordered sequence of one or more ISLs, i.e., $p = (e_1, \ldots, e_{|p|})$, and different path may partially share the same ISLs. If D_h bits are transmitted, ELARA splits them across multiple such paths, and path p receives fraction $\lambda_p^h \in [0,1]$:

$$d_p^h = \lambda_p^h D_h, \qquad \sum_{p \in \mathcal{P}_{r,i}^h} \lambda_p^h = 1, \qquad 0 < D_h \le B_{r,i}^h. \tag{10}$$

The aggregate load on shared ISL $e \in p$ is formulated by:

$$f_e^h = \sum_{p \in \mathcal{P}_{r,i}^h : e \in p} d_p^h, \qquad 0 \le f_e^h \le R_e(t_h) \bar{\Delta}_h. \tag{11}$$

For a multi-hop path p, the subflow completion time follows the link-delay model in (5):

$$\delta_p^h = \sum_{e \in p} T_e^{\mathrm{cm}}(d_p^h, t_h), \qquad \forall p \in \mathcal{P}_{r,i}^h. \tag{12}$$

Since the selected paths transmit in parallel, the effective phase duration is determined by the slowest active path:

$$\Delta_h^{\mathrm{ru}} = \max_{p \in \mathcal{P}_{r,i}^h} \delta_p^h, \quad \text{and,} \quad \Delta_h^{\mathrm{ru}} \le \bar{\Delta}_h. \tag{13}$$

The phase energy is the sum of transmission energy over all traffic-carrying ISLs:

$$E_{r,i}^{h,\mathrm{ru}} = \sum_{e \in \mathcal{E}(t_h)} P_e^{\mathrm{tx}} \frac{f_e^h}{R_e(t_h)}. \tag{14}$$

After phase h, the residual data and decision epoch are updated as:

$$B_{r,i}^{h+1} = B_{r,i}^{h} - D_h, \qquad \tau_{r,i}^{h+1} = \tau_{r,i}^{h} + \Delta_h^{\text{ru}}. \tag{15}$$

If $B_{r,i}^{h+1} > 0$, the next phase is solved under the updated topology, path set, and link rates. For $H_{r,i}$ phases, the routing latency and energy from $u_{r,i}$ to $v_{r,i}$ are:

$$T_{r,i}^{\text{ru}} = \sum_{h=1}^{H_{r,i}} \Delta_h^{\text{ru}}, \qquad E_{r,i}^{\text{ru}} = \sum_{h=1}^{H_{r,i}} E_{r,i}^{h,\text{ru}}, \tag{16}$$

The selected replica receives the data at $\tau_{r,i}^{\text{arr}} = \tau_{r,i} + T_{r,i}^{\text{ru}}$. The selected satellite then executes m_i, whose computation delay and energy are given by (6) and (7), respectively.

The cumulative execution latency and energy of request r are given by:

$$T_r^{\text{e2e}} = \sum_{i=1}^{L+1} T_{r,i}^{\text{ru}} + \sum_{i=1}^{L} T_{m_i}^{\text{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\text{arr}})\big), \tag{17}$$

$$E_r^{\text{tot}} = \sum_{i=1}^{L+1} E_{r,i}^{\text{ru}} + \sum_{i=1}^{L} E_{m_i}^{\text{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\text{arr}})\big), \tag{18}$$

where $i = L+1$ denotes the final output delivery to v_d.

b) Service Replica Migration Model: Let $\mathbf{X}(t) = [x_{m,v}(t)]$ represent the microservice replica deployment matrix at slot t. ElARA periodically updates the deployment matrix every few time slots by selecting an action $a_{m_i} \in \{\text{add}, \text{remove}, \text{move}, \text{no-op}\}$ for each m_i in a microservice subset $\mathcal{M}'(t) \subset \mathcal{M}$ at time slot t. Then we have:

$$\mathbf{X}(t+1) = \Phi(\mathbf{X}(t), \mathcal{A}_t), \tag{19}$$

where $\Phi(\cdot)$ denotes the deployment transition induced by the selected action set $\mathcal{A}_t = \{a_{m_i} | \forall m_i \in \mathcal{M}'(t)\}$. All redeployment actions should preserve the replica-count and storage constraints in (8) and (9).

IV. Problem Formulation of ELARA

We formulate ELARA as an online energy- and latency-aware service routing problem over a dynamic SCPN. The objective is to simultaneously minimize the end-to-end execution latency and total energy consumption of service chains, including both communication and computation costs, while accounting for the overhead of replica redeployment.

A. Problem Formulation

For each incoming request, ELARA first selects the serving satellite v for each microservice m_i, which satisfies:

$$\sum_{v \in \mathcal{V}} y_{r,i,v} = 1, \quad y_{r,i,v} \le x_{m_i,v}\big(s(\tau_{r,i})\big), \quad \forall r, i, v. \tag{20}$$

Let $v_{r,i}$ denote the selected satellite satisfying $y_{r,i,v_{r,i}} = 1$. ELARA then makes the cross-slot routing policy for service-routing segment i:

$$\Pi_{r,i}^{\text{ru}} = \{D_h, \lambda_p^h, f_e^h, \Delta_h^{\text{ru}}\}_{h=1}^{H_{r,i}}, \tag{21}$$

which follows the multi-hop routing model in Sec. III-D. Additionally, ELARA periodically updates the deployment matrix by applying $\mathcal{A}(t), \forall t \in \mathcal{T}'$, where $\mathcal{T}' \subset \mathcal{T}$.

Following (17) and (18), given an online request r, the request-level optimization objective is defined as:

$$\begin{aligned} \mathscr{P}: \ \underset{y_{r,v}, \Pi_r^{\text{ru}}, \mathcal{A}}{\text{minimize}} \quad & J_r = \alpha T_r^{\text{e2e}} + \beta E_r^{\text{tot}} + \rho \mathbb{I}_r^{\text{fail}} \\ \text{subject to} \quad & (20), (21), && \text{(C1)} \\ & (10), (11), (13), (15), && \text{(C2)} \\ & (8), (9), (19), && \text{(C3)} \end{aligned}$$

where α, β, and ρ are non-negative weights, and $\mathbb{I}_r^{\text{fail}}$ indicates whether request r violates its deadline. Constraint C1 enforces serving satellite selection by assigning each microservice stage to exactly one satellite hosting a valid replica, and defining the corresponding routing policy for each service routing segment. Constraint C2 ensures feasible cross-slot multi-hop routing under time-varying topology by enforcing path splitting, shared ISL capacity, phase duration, and residual data update constraints. Constraint C3 restricts adaptive replica redeployment to states that satisfy both replica-count and onboard storage constraints after each migration action.

B. Problem Hardness

Problem $\mathscr{P}$ is NP-hard. Even in a single-slot static case with fixed deployment and fixed link and computing capacities, $\mathscr{P}$ requires selecting replicas for a service chain and connecting them through capacity-constrained paths. This restricted case contains the placement of service function chains and constrained routeing as special cases and is therefore NP-hard. The full problem further couples these decisions with cross-slot routeing and adaptive replica redeployment, motivating ELARA's online decomposition.

V. The Proposed ELARA Framework

This section presents ELARA, an online framework that coordinates request-time microservice execution and periodic replica redeployment over dynamic SCPNs. ELARA has two interacting layers. The fast layer makes per-request decisions: for each microservice stage, it selects a serving satellite from valid replicas and routes intermediate data over time-varying multi-hop ISLs. The slow layer periodically adjusts replicas by exploiting recent request patterns, route failures, and execution feedback. Both layers share the same latency-energy cost model in Sec. III, so routing, computation, and migration are optimized under consistent system states.

A. Request-Time Orchestration

For an incoming request r, ELARA initializes the current node and time as $u \leftarrow v_s$ and $\tau \leftarrow \tau_{\text{in}}$. Then it executes the requested microservices sequentially. At stage i, the controller first constructs the candidate set $\mathcal{C}_{r,i} \subseteq \mathcal{R}_{m_i}(t)$ from satellites hosting m_i. To reduce the action space, ELARA keeps at most K candidates ranked by hop distance and compute queueing delay. The GNN-PPO policy then selects one serving satellite $v_{r,i} \in \mathcal{C}_{r,i}$. After that, the routing module transfers

Algorithm 1: ELARA Request-Time Orchestration

Input : Request $r = \langle v_s, m_1, \ldots, m_L, v_d, \tau_{\text{in}} \rangle$, topology snapshots $\{\mathcal{G}(t)\}$, deployment matrix $\mathbf{X}(t)$

Output: Serving satellites, routing paths, delay and energy

$u \leftarrow v_s$, $\tau \leftarrow \tau_{\text{in}}$, $T_r \leftarrow 0$, $E_r \leftarrow 0$;
for $i = 1$ **to** L **do**
  Obtain data size $B_{r,i}$ and candidate replicas $\mathcal{C}_{r,i} \subseteq \mathcal{R}_{m_i}(t)$;
  $v_{r,i} \leftarrow$ **GNN-PPO-Select**$(r, i, u, \tau, \mathcal{C}_{r,i})$;
  if $v_{r,i}$ *does not exist* **then**
    mark r as failed and stop;
  end
  $\Pi_{r,i}, T^{\text{ru}}_{r,i}, E^{\text{ru}}_{r,i} \leftarrow$ **SlotFlowRoute**$(u, v_{r,i}, B_{r,i}, \tau)$;
  if *routing fails* **then**
    mark r as failed and stop;
  end
  Compute m_i on $v_{r,i}$ and obtain $T^{\text{cp}}_{r,i}$, $E^{\text{cp}}_{r,i}$;
  $T_r \leftarrow T_r + T^{\text{ru}}_{r,i} + T^{\text{cp}}_{r,i}$, $E_r \leftarrow E_r + E^{\text{ru}}_{r,i} + E^{\text{cp}}_{r,i}$;
  $u \leftarrow v_{r,i}$, $\tau \leftarrow \tau + T^{\text{ru}}_{r,i} + T^{\text{cp}}_{r,i}$;
  Record local reward and PPO transition;
end
Route final output from u to v_d and update T_r, E_r;
Record terminal reward and update request result;

the required data from u to $v_{r,i}$ and the onboard computing module executes m_i after the data arrives. Finally, u and τ are updated to the selected satellite and the completion time of this stage. After all stages finish, ELARA routes the output data to the destination satellite v_d. The whole procedure is summarized in Algorithm 1.

B. GNN-PPO Serving Satellite Selection

The serving satellite decision depends on the current topology, onboard computing state, replica placement, and the remaining request context. ELARA encodes the current snapshot $\mathcal{G}(t)$ as a graph whose node feature for satellite v is

$$\mathbf{x}_v(t) = [\bar{f}_v, \bar{q}_v(t), \bar{s}_v(t), \bar{p}_v, \bar{n}_v, \mathbb{I}_{v=u}, \mathbb{I}_{v=v_d}, \mathbb{I}_{m_i \in v}, \bar{d}_v(t), \mu_v(t), \xi_v(t), \ell_v(t)]. \tag{23}$$

where the entries denote normalized CPU frequency, compute queue delay, hosted-service count, orbital-plane index, intra-plane position, current-node indicator, destination indicator, current-service replica indicator, node degree, compute utilization, CPU discount factor, and compute-load state, respectively. The request-stage context is encoded as

$$\mathbf{c}_{r,i} = [\bar{L}_r, \bar{i}, \overline{L_r - i}, \bar{w}_{m_i}, \bar{z}_{m_i}, \bar{B}_r], \tag{24}$$

which captures the request length, stage index, remaining stages, workload of m_i, service image size, and total data volume.

ELARA applies message passing over $\mathcal{G}(t)$ to obtain node embeddings $\mathbf{h}_v$. For each candidate $v \in \mathcal{C}_{r,i}$, the policy further appends an auxiliary feature vector $\mathbf{e}_v$ containing normalized route delay, future route-failure risk, bottleneck/egress shortage, and estimated computing cost. The candidate score is produced by

$$g_\theta(v) = \text{MLP}_\theta\big([\mathbf{h}_v, \mathbf{h}_u, \mathbf{h}_{v_d}, \mathbf{z}_{r,i}, \mathbf{e}_v]\big), \tag{25}$$

where $\mathbf{z}_{r,i}$ is the embedded request-stage vector. A masked softmax is then applied only over legal candidates:

$$\pi_\theta(v|s_{r,i}) = \frac{\exp(g_\theta(v))}{\sum_{v' \in \mathcal{C}_{r,i}} \exp(g_\theta(v'))}. \tag{26}$$

During training, ELARA samples from π_θ; during inference, it selects the candidate with the largest probability. The value head uses the current-node, destination-node, and request-stage embeddings to estimate the state value. The local reward of stage i is

$$r^{\text{loc}}_{r,i} = -\big(\alpha T^{\text{step}}_{r,i} + \beta E^{\text{step}}_{r,i} + \lambda N^{\text{cross}}_{r,i}\big), \tag{27}$$

where $T^{\text{step}}_{r,i}$ and $E^{\text{step}}_{r,i}$ include routing and computing costs, and $N^{\text{cross}}_{r,i}$ is the number of crossed slots. After a request terminates, ELARA distributes a discounted terminal signal to its stage transitions and updates the policy using clipped PPO with generalized advantage estimation.

C. Time-Slot-Aware Multi-Path Routing

Given source u, target v, data size D, and start time τ, ELARA routes data across a finite horizon of future topology slots. In each slot, it builds a residual graph using the current ISL set. For each edge $e = (a, b)$, the residual capacity is computed from the remaining slot time and the effective rate $R_e(t)$ after discounting background traffic. The unit cost of sending one data unit over e is

$$c_e(t) = \alpha(\nu^{\text{tr}}_e + \nu^{\text{pr}}_e + \nu^{\text{q}}_e + \nu^{\text{sw}}) + \beta E^{\text{tr}}_e + \rho \Gamma_e(t), \tag{28}$$

where $\Gamma_e(t)$ is the future link-failure risk estimated over a short look-ahead horizon. ELARA repeatedly augments flow along the minimum-cost residual path until either the data is delivered, no feasible path remains, or the augmentation limit is reached. The positive flows are decomposed into multiple paths and transmitted in parallel within the slot. If only part of the data is delivered before the slot boundary, the remaining data is carried into the next topology snapshot and re-routed. In implementation, each route record stores the delivered data volume, slot index, path-level delay components, communication energy, bottleneck capacity, and future failure risk, which are reused by the serving-satellite selector and the training feedback.

D. Bandit-Based Replica Redeployment

The slow layer adapts replica deployment every several slots. ELARA first aggregates recent execution feedback and computes a pressure score for each microservice:

$$\psi_m = n_m + \omega_1 \bar{T}^{\text{route}}_m + \omega_2 \bar{T}^{\text{wait}}_m + \omega_3 F_m + \omega_4 T^{95}_m + \omega_5 U_m, \tag{29}$$

where n_m is the request count, $\bar{T}_m^{\text{route}}$ is average routing delay, $\bar{T}_m^{\text{wait}}$ is average compute waiting time, F_m is the failure count, T_m^{95} is the 95-th percentile stage delay, and U_m is replica-utilization imbalance. Only the top pressured services are considered in each redeployment round. Instead of treating each service-satellite pair as an arm, ELARA defines an arm as an orbital-level action

$$a = (\text{type}, m, p_s, p_t), \quad \text{type} \in \{\text{add}, \text{remove}, \text{move}, \text{no-op}\}. \tag{30}$$

This design greatly reduces the decision space. After an arm is selected, ELARA resolves it to concrete source and target satellites by considering replica failure observations, storage capacity, satellite service load, and neighboring link quality. For add and move actions, the service image is transferred through the same routing module, and the migration cost includes routing latency, communication energy, slot crossings, link-failure risk, image size, and startup overhead. An action is applied only if its estimated reward exceeds a safety margin. The bandit selection follows UCB:

$$\text{UCB}(a) = \bar{R}_a + c\sqrt{\frac{\ln N}{N_a}}, \tag{31}$$

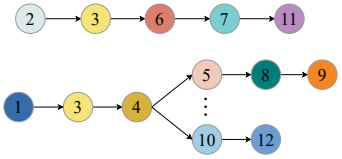
where $\bar{R}_a$ is the empirical reward, N_a is the number of pulls of arm a, and N is the total number of pulls. Overall, the per-stage serving decision costs $\mathcal{O}(L_g(|\mathcal{V}| + |\mathcal{E}|)d_h + Kd_h)$, the cross-slot routing costs $\mathcal{O}(HA|\mathcal{E}|\log|\mathcal{V}|)$ for horizon H and augmentations A, and the orbital-level redeployment evaluates only $\mathcal{O}(K_s K_p|\mathcal{P}|)$ arms per round.

VI. Performance Evaluation

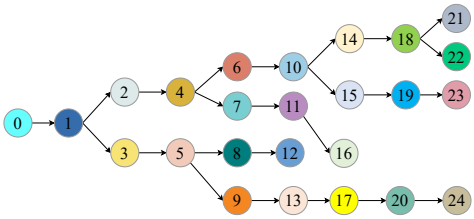
A. Experiment Setup

Constellation. In experiments, we consider a 72/12/1 Telesat Star Constellation system, which consists of $P = 6$ polar orbit planes whose inclination is $99.5°$, and the orbital altitude is $h = 1000km$ on the ground [33].

Satellite Computing Capability. We suppose LEO satellites are equipped with several onboard processors and storage components. According to [19], we assign the satellite computing capability following a uniform distribution, as $\varphi(v_k) \sim \text{DiscreteUniform}(\{100, 160, 220, 280, 340, 400\})$, while the processor power $\Psi(v_k) = \varphi(v_k) \times 0.01$ in Watts.



(a) Chained and Treed topologies.



(b) Sophisticated-Treed (Soph-Treed) topology.

Fig. 1: The overview of service request topologies from terrestrial stations.

ISL Parameters. Similar to settings in [34]–[36], the transmission power of all relevant satellites is standardized at $P_T = 3$ (Watt). In the context of the Telesat Constellation, which achieve inter-satellite interconnection across free-space optical link, the carrier frequency of each of satellite is set as f_c=193(THz), , and the corresponding carrier wavelength is λ=1550(nm) while the channel bandwidth between two satellites is defined as BW=2%f_c. Besides, we specify the antenna diameter transceiver as D_R=6(mm), the pointing loss angle $\theta_0 = 0.01\pi$ and the optical conversion efficiency of the transceiver modules $\eta = 0.8$.

Microservices. According to the microservice setting adopted by [19], we heuristically construct the microservices testbed as follows: There are $|\mathcal{M}| = 60$ microservices whose data volume follows a Gaussian distribution with a mean of 50 and variance of 4 in Gbits. For each microservice, there are $4 \sim 6$ replicas distributed on satellites, while each satellite can concurrently host at most $\lfloor|\mathcal{M}| \times 6/72\rfloor$ microservices.

Ablation Solutions: We design the following benchmarks to evaluate the performance of our proposed algorithm.

1) **Random:** It randomly selects a satellite for each microservice and uses the Dijkstra algorithm to get the shortest path when dealing with relay links.
2) **Computation greedy (Comp-greedy):** It chooses the satellite that offers the minimum computation energy cost for executing each microservice. The relay routing policy is the same as **Random** policy.
3) **Communication greedy (Comm-greedy):** It picks the satellite pair with the minimum communication energy cost, including the potential relay links.

We define three service request topologies: the Chained topology, the multi-forked Treed topology, and the Sophisticated Treed (Soph Tree) topology, as depicted in Fig. 1.

To verify the effectiveness of our algorithm, we conducted a series of experiments on a space topology comprising 72 satellites, each hosting 60 microservices with 4∼6 replicas. We generated 100 service request chains, varying in length from 10 to 20, to simulate a chained microservice topology. We also constructed 100 random treed topologies, containing 20 to 45, featuring a stochastic number of leaf nodes and varying branch configurations. For each sophisticated treed topology, microservices were randomly assigned to nodes, and this assignment process was independently repeated 100 times. For each topology, energy cost data was collected from 100 samples. The final results were derived by averaging the energy cost values across these samples. It ensures robust and reliable performance evaluation by mitigating variations caused by randomness in the topology generation and microservice allocation processes.

B. Experimental Results

1) Effectiveness: As illustrated in Fig. 2(a), our proposed Steiner consistently outperforms all baseline methods across various microservices topology scenarios, and its advantages become more pronounced in more complex topologies. This enhanced performance is charged to our comprehensive approach that simultaneously considers both computation and communication costs when determining the most efficient routing paths. The comp-greedy method outperforms the random method by prioritizing minimizing computation costs. The comm-greedy method surpasses both comp-greedy and random methods by focusing on minimizing communication costs. These findings emphasize the necessity of accounting for both computation and communication factors to design effective and energy-conscious routing strategies for the SCPN.

2) Impact of constellation scales: To evaluate the scalability of our algorithm, we systematically varied the number of satellites in the constellation by adjusting the number of orbits while maintaining consistent microservice deployment strategies. Experiments were conducted in two constellation settings separately, as illustrated in Fig. 2(b) and Fig. 2(c). These configurations allowed us to assess the algorithm's performance under varying levels of network complexity and service topology. Our results demonstrate that the proposed approach consistently outperforms alternative methods across all tested constellation scales and topological scenarios. Notably, its effectiveness becomes increasingly pronounced as the network complexity escalates, highlighting its robustness and adaptability to intricate environments.

3) Impact of transmitter power: Given the dominant role of communication costs in the total energy consumption of satellite systems, we conducted a series of experiments to explore how varying channel conditions influence the performance of our Steiner scheme. Specifically, we experimented with four different transmitter power levels on satellites across diverse microservice request schemes to examine their effect on energy efficiency. The results shown in Figs. 3(a), 3(b) and 3(c) indicate that as transmitter power increases, the average energy cost shows an upward trend. This trend highlights the trade-off between signal strength and energy consumption, with higher power levels leading to greater energy expenditure. Despite this increase in energy cost, our Steiner algorithm consistently outperformed other benchmark strategies across all service request scenarios. This superior performance is attributed to the algorithm's ability to dynamically adapt to the complexities of space network topologies and fluctuating channel conditions. By optimizing service routing paths, our approach minimizes unnecessary energy use under varying channel conditions. These findings underscore the importance of intelligent service routing mechanisms in the SCPN, particularly in environments with variable communication links.

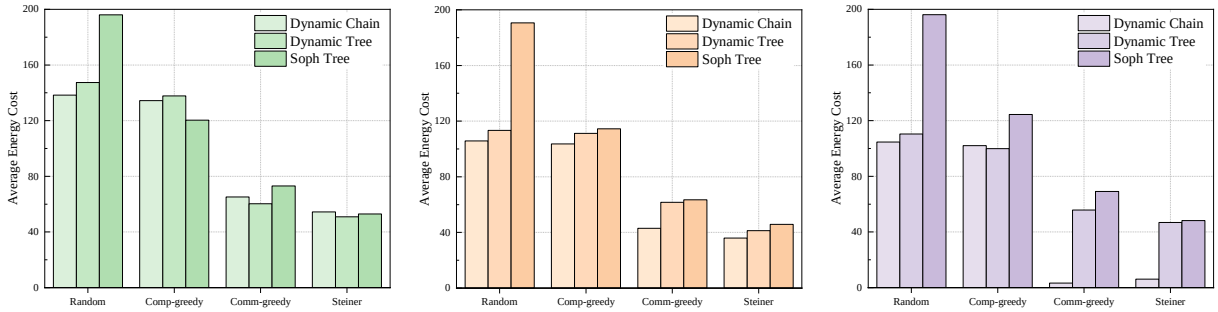
4) Impact of faulty satellites: Given the constraints of limited power supply and the effects of solar eclipses, some satellites may need to enter a low-power mode, functioning solely as relay nodes for message forwarding [17]. To evaluate the robustness of our proposed method under such conditions, we conducted experiments by randomly disabling a subset of satellites in a constellation of 72 satellites. Specifically, we tested scenarios with 10, 20, and 25 faulty satellites by eliminating their microservices, as shown in Figs 4(a), 4(b), 4(c). The results reveal that energy cost generally increases with an increasing number of faulty satellites for all methods. This trend can be attributed to the reduced availability of active nodes, which forces longer transmission paths and higher energy consumption for maintaining connectivity (much more relay satellites are required for message forwarding). Despite this challenge, Steiner invariably outperforms other methods in all scenarios. Its superior performance is attributed to its robust design, which can effectively adapt to changes in network topology and operational conditions. By optimizing power allocation and service routing decisions, the algorithm minimizes disruptions caused by satellite failures.

5) Impact of microservice deployment schemes: To investigate the influence of microservice deployment and scaling strategies on energy consumption with in the SCPN, we conducted experiments by varying the number of replicas allocated for each microservice. Specifically, we tested three replica ranges: $[2, 6]$, $[4, 6]$, and $[6, 10]$. And the experimental results, illustrated in Figs. 5(a), 5(b), and 5(c), reveal that energy costs generally decreases as the scale of microservice deployment expands. This trend is attributed to the increased availability of service nodes and routing paths, which provide more opportunities for route optimization and efficient power utilization. Our algorithm regularly demonstrates superior performance in reducing energy costs compared to other benchmarks, particularly in more complex microservice-satellite patterns. This superior performance can be attributed to the algorithm's ability to leverage the increased number of service routing paths and nodes, enabling more advanced route strategies. The expanded deployment range provides greater flexibility in service selections and routing decisions, allowing the algorithm to identify more energy-efficient configurations. By contrast, the energy costs associated with routing paths generated by random and comp-greedy methods occasionally increased as the number of replicas grew. This counterintuitive behavior can be attributed to inefficient routing decisions caused by the higher number of service nodes and calling paths, which complicate the decision-making process and lead to suboptimal resource utilization.

6) Adaptability and stability: Furthermore, as depicted in Fig 6, which presents the energy distribution derived from 300 independent experiments, underscores the superior stability of the Steiner policy compared to other benchmark strategies. The results reveal that, even under sophisticated treed topologies with heightened complexity, the Steiner policy exhibits less variance in energy costs. This stability highlights its robustness and adaptability in managing diverse network conditions and service requirements.

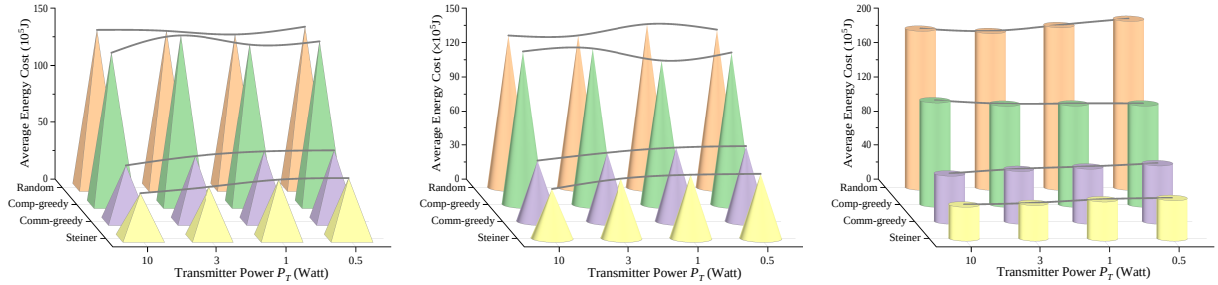
VII. Conclusion

In this paper, we proposed a DST-based solution to provide an energy-efficient service routing scheme for tasks from



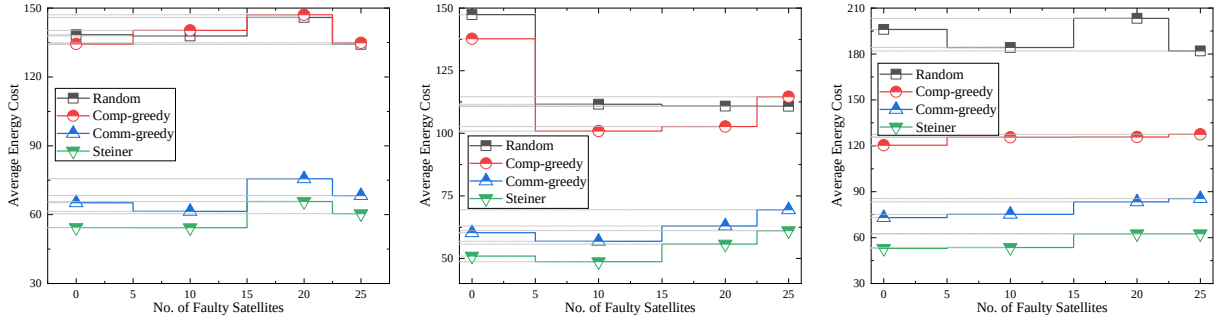
(a) Constellation with 72 satellites. (b) Constellation with 48 satellites. (c) Constellation with 108 satellites.

Fig. 2: Performance comparison on energy-cost under varying constellation scales for dynamic microservice request topologies.



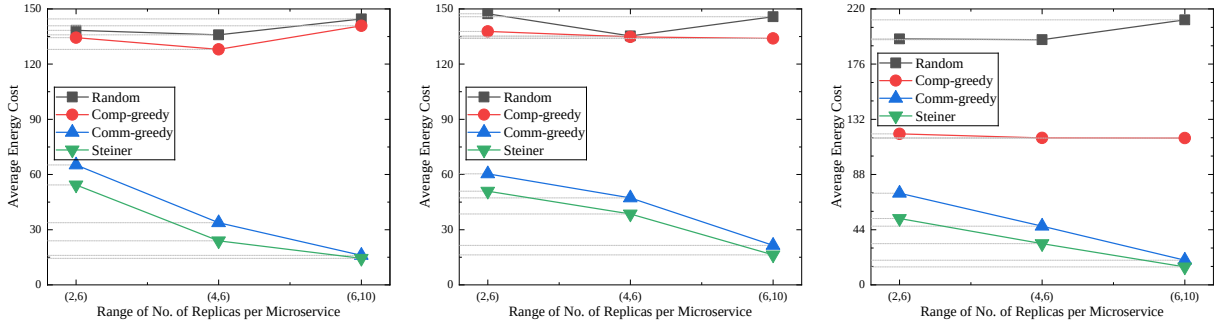
(a) Chained service topology. (b) Treed service topology. (c) Sophisticated Treed service topology.

Fig. 3: Performance comparison on energy-cost under different onboard transmitter power configurations for dynamic microservice request topologies.



(a) Robustness over faulty satellites (Chain). (b) Robustness over faulty satellites (Tree). (c) Robustness over faulty satellites (Soph Tree).

Fig. 4: Performance comparison on energy-cost under different configurations on number of faulty satellites for dynamic service request topologies.



(a) Microservice scale (Chain). (b) Microservice scale (Tree). (c) Microservice scale (Soph Tree).

Fig. 5: Performance comparison on energy-cost under different configurations on number of replicas of microservices for dynamic service request topologies.

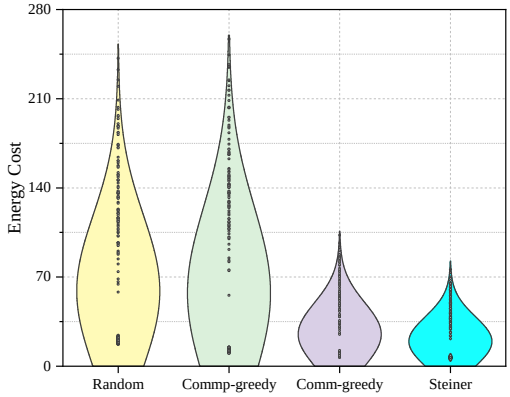


Fig. 6: Energy distribution of Soph Tree.

terrestrial clients. We systematically developed the SCPN models, including communication and computation, covering various aspects of the LEO satellite network. We also mathematically formulated the energy optimization problem for satellite-microservice routing and transformed it into finding a feasible DST solution within an augmented directed satellite-microservice colocated graph. To approximate the optimal solution, we designed a series of heuristic algorithms. Experimental results show that our algorithm significantly outperforms other benchmarks in energy efficiency, robustness, and adaptability across different network and microservice configurations in the context of SCPN.

REFERENCES

[1] Z. Xiao, J. Yang, T. Mao, C. Xu, R. Zhang, Z. Han, and X.-G. Xia, "Leo satellite access network (leo-san) towards 6g: Challenges and approaches," *IEEE Wireless Communications*, 2022.

[2] S. Ma, Y. C. Chou, H. Zhao, L. Chen, X. Ma, and J. Liu, "Network characteristics of leo satellite constellations: A starlink-based measurement from end users," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pp. 1–10, 2023.

[3] H. Al-Hraishawi, H. Chougrani, S. Kisseleff, E. Lagunas, and S. Chatzinotas, "A survey on nongeostationary satellite systems: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 101–132, 2023.

[4] B. Zhang, Y. Wu, B. Zhao, J. Chanussot, D. Hong, J. Yao, and L. Gao, "Progress and challenges in intelligent remote sensing satellite systems," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 1814–1822, 2022.

[5] Y. Li, M. Wang, K. Hwang, Z. Li, and T. Ji, "Leo satellite constellation for global-scale remote sensing with on-orbit cloud AI computing," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, vol. 16, pp. 9369 – 9381, 2023.

[6] Z. Zhai, L. Zeng, T. Ouyang, S. Yu, Q. Huang, and X. Chen, "Seco: Multi-satellite edge computing enabled wide-area and real-time earth observation missions," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 2548–2557, 2024.

[7] I. Leyva-Mayorga, M. Martinez-Gost, M. Moretti, A. Pérez-Neira, M. Á. Vázquez, P. Popovski, and B. Soret, "Satellite edge computing for real-time and very-high resolution earth observation," *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 6180–6194, 2023.

[8] Q. Ouyang, N. Ye, J. Gao, A. Wang, and L. Zhao, "Joint in-orbit computation and communication for minimizing download time from leo satellites," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3950–3963, 2024.

[9] Z. Shen, J. Jin, C. Tan, A. Tagami, S. Wang, Q. Li, Q. Zheng, and J. Yuan, "A survey of next-generation computing technologies in space-air-ground integrated networks," *ACM Comput. Surv.*, vol. 56, aug 2023.

[10] B. Shang, Y. Yi, and L. Liu, "Computing over space-air-ground integrated networks: Challenges and opportunities," *IEEE Network*, vol. 35, no. 4, pp. 302–309, 2021.

[11] C. Wang, Y. Zhang, Q. Li, A. Zhou, and S. Wang, "Satellite computing: A case study of cloud-native satellites," in *2023 IEEE International Conference on Edge Computing and Communications (EDGE)*, pp. 262–270, 2023.

[12] S. Ye, J. Du, L. Zeng, W. Ou, X. Chu, Y. Lu, and X. Chen, "Galaxy: A resource-efficient collaborative edge ai system for in-situ transformer inference," *arXiv preprint arXiv:2405.17245*, 2024.

[13] D. Merkel *et al.*, "Docker: lightweight linux containers for consistent development and deployment," *Linux j*, vol. 239, no. 2, p. 2, 2014.

[14] T. Salah, M. J. Zemerly, C. Y. Yeun, M. Al-Qutayri, and Y. Al-Hammadi, "The evolution of distributed systems towards microservices architecture," in *2016 11th International Conference for Internet Technology and Secured Transactions (ICITST)*, pp. 318–325, IEEE, 2016.

[15] J. Wang, "Towards geoai as a containerized microservice," in *Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems*, pp. 1–2, 2023.

[16] A. Ruiz-Zafra, J. Pigueiras-del Real, M. Noguera, L. Chung, D. G. Barres, and K. Benghazi, "Servitization of customized 3d assets and performance comparison of services and microservices implementations," *IEEE Transactions on Services Computing*, vol. 17, no. 1, pp. 194–208, 2024.

[17] Y. Yang, M. Xu, D. Wang, and Y. Wang, "Towards energy-efficient routing in satellite networks," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3869–3886, 2016.

[18] Y. Zhang, Q. Wu, Z. Lai, Y. Deng, H. Li, Y. Li, and J. Liu, "Energy drain attack in satellite internet constellations," in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pp. 1–10, 2023.

[19] J. Cao, S. Zhang, Q. Chen, H. Wang, M. Wang, and N. Liu, "Computing-aware routing for leo satellite networks: A transmission and computation integration approach," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 12, pp. 16607–16623, 2023.

[20] Z. Yu, Y. Jiang, X. Liu, Y. Shi, C. Jiang, and L. Kuang, "Microservice deployment in space computing power networks via robust reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 25, no. 2, pp. 2382–2398, 2026.

[21] Y. Gao, Z. Yan, K. Zhao, T. de Cola, and W. Li, "Joint optimization of server and service selection in satellite-terrestrial integrated edge computing networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 2, pp. 2740–2754, 2024.

[22] Q. Xia, G. Wang, Z. Xu, W. Liang, and Z. Xu, "Efficient algorithms for service chaining in nfv-enabled satellite edge networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5677–5694, 2024.

[23] H. G. Abreha, H. Chougrani, I. Maity, Y. Drif, C. Politis, and S. Chatzinotas, "Fairness-aware vnf mapping and scheduling in satellite edge networks for mission-critical applications," *IEEE Transactions on Network and Service Management*, vol. 21, no. 6, pp. 6716–6730, 2024.

[24] X. He, T. Wang, Y. Shi, and X. Liu, "Throughput-optimized service routing for microservice flows in leo satellite networks," *IEEE Transactions on Mobile Computing*, vol. 25, no. 5, pp. 7196–7209, 2026.

[25] D. Ron, F. A. Yusufzai, S. Kwakye, S. Roy, N. Sastry, and V. K. Shah, "Time- dependent network topology optimization for leo satellite constellations," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, pp. 1–10, 2025.

[26] Y. Yu, J. Yang, C. Guo, H. Zheng, and J. He, "Joint optimization of service request routing and instance placement in the microservice system," *Journal of Network and Computer Applications*, vol. 147, p. 102441, 2019.

[27] K. Peng, L. Wang, J. He, C. Cai, and M. Hu, "Joint optimization of service deployment and request routing for microservices in mobile edge computing," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 1016–1028, 2024.

[28] R. Xie, Q. Tang, Q. Wang, X. Liu, F. R. Yu, and T. Huang, "Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues," *IEEE Network*, vol. 34, no. 3, pp. 224–231, 2020.

[29] X. Zhang, J. Liu, R. Zhang, Y. Huang, J. Tong, N. Xin, L. Liu, and Z. Xiong, "Energy-efficient computation peer offloading in satellite edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, pp. 3077–3091, 2024.

[30] W. Liu, H. Yang, and J. Li, "Multi-functional time expanded graph: A unified graph model for communication, storage and computation

for dynamic networks over time," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 418–431, 2023.

[31] Y. Li, Q. Zhang, H. Yao, R. Gao, X. Xin, and F. R. Yu, "Stigmergy and hierarchical learning for routing optimization in multi-domain collaborative satellite networks," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 5, pp. 1188–1203, 2024.

[32] J. Zhu, Y. Shi, Y. Zhou, C. Jiang, and L. Kuang, "Hierarchical learning and computing over space-ground integrated networks," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10423–10440, 2025.

[33] I. Del Portillo, B. G. Cameron, and E. F. Crawley, "A technical comparison of three low earth orbit satellite constellation systems to provide global broadband," *Acta astronautica*, vol. 159, pp. 123–135, 2019.

[34] S. Nie and I. F. Akyildiz, "Channel modeling and analysis of inter-small-satellite links in terahertz band space networks," *IEEE Transactions on Communications*, vol. 69, no. 12, pp. 8585–8599, 2021.

[35] I. Leyva-Mayorga, B. Soret, and P. Popovski, "Inter-plane inter-satellite connectivity in dense leo constellations," *IEEE Transactions on Wireless Communications*, vol. 20, no. 6, pp. 3430–3443, 2021.

[36] A. Polishuk and S. Arnon, "Optimization of a laser satellite communication system with an optical preamplifier," *J. Opt. Soc. Am. A*, vol. 21, pp. 1307–1315, Jul 2004.